

本地 RAG 本文（展版）

面向 agent-lab / 本地检索实验 / 私有语料评测
版本 2.0 · 合成内部文档风格

使用说明

- 这份文档不是通用百科，而是刻意写成“像真实团队内部资料”的技术长文，用于测试本地 RAG 在私有知识上的召回、切块、重排与答案归因能力。
- 文中出现的项目名、组件名、事件名、规则编号、目录结构和排障口径均为合成内容，目标是降低本地通用模型凭预训练常识直接作答的概率。
- 适合用于关键词检索、混合召回、chunk preview、top_k、rerank、引用片段抽取与多跳问答实验。

1. 为什么要把样本文档写得“像内部资料”

很多人在做本地 RAG 时，第一批测试语料喜欢直接拿公开博客、开源框架介绍、数据库教程或 GPU 对比文章。这样做上手快，但很快会遇到一个误区：本地模型即使不检索，也能凭参数记忆给出一段看似合理的回答。于是你分不清当前系统回答得对，到底是因为真的从知识库里命中了片段，还是因为模型本来就知道这些内容。为了避免这个误判，样本文档最好带有明显的“私有性”，也就是包含不会出现在公开语料里的命名、流程、异常口径与约定。

在这份扩展版文档里，我们刻意引入了一组并不存在于公开框架中的术语，例如“霜桥索引目录”“灰瓦快照”“回声段合并”“蓝门校验”“七号通道回填”“南侧预热表”“静夜写入窗”等。对通用模型而言，这些词没有稳定语义；但对一个接入了本地检索的系统而言，它们会在 chunk 中反复出现、建立局部上下文，从而形成可以被召回和引用的私有知识网。

你在评测时可以故意问一些只有这份文档才包含答案的问题，例如：灰瓦快照在什么情况下必须清空？七号通道回填与普通补录的区别是什么？静夜写入窗默认从几点开始？这些问题即使表面简单，如果检索没有命中、引用没有带对、rerank 没把相关 chunk 排上来，本地模型往往会开始编造。这样一来，文档本身就成了识别幻觉和识别“真检索”能力的试金石。

2. 一个适合学习的最小骨架：薄入口、厚服务、可落盘

延续原始 demo 的思想，一个真正适合学习的项目，入口层必须尽量薄。FastAPI 里的 main.py 不应该塞进 loader、切块、索引、搜索、问答、权限判断和日志格式化的所有逻辑。它最好的状态，是只做应用初始化、路由挂载、异常处理和配置装配。请求进来后，路由层只负责把 HTTP 参数转成稳定的数据结构，再交给 service 层处理。这样你未来不管要不要换成 CLI、批处理脚本、定时任务甚至桌面应用，核心能力都不需要重写。

在一个用于本地实验的小系统里，最容易被忽视的不是“能不能跑”，而是“结果能不能看”。所以这里建议保留 JSON 落盘，而不要急着把所有中间结果塞进数据库。原因很简单：你需要能肉眼审查 ingest 的产出。比如 data/processed/night-orchid-01.json 里应该能直接看到 source、doc_hash、chunk_id、offset_start、offset_end、overlap_from、overlap_to、section_path、paragraph_no、preview_seed 等字段。只要这些字段足够清晰，你遇到召回异常时就能快速定位：到底是文本没读出来、切块丢了边界、索引没更新，还是排序逻辑错了。

有些学习者喜欢一上来就把项目接进向量库、队列系统、云端 embedding API 和前端聊天界面。表面上功能很全，实际上可解释性迅速下降。一旦 ask 接口答错，你很难判断它错在 embedding 维度、索引状态、召回窗口、prompt 组装还是模型生成。相反，一个“薄入口、厚服务、可落盘”的骨架虽然朴素，却能让你保持对每层行为的观察权，这比一开始追求花哨功能更重要。

3. Loader 设计：读取不是小事，错误信息更不是附属品

loader 的职责不是“把文件读成字符串”这么简单，它实际上承担了语料进入系统的第一道质量门。txt 通常最省心，但也会遇到 UTF-8 与 GBK 混用、混入零宽字符、日志文件里存在不可见分隔符等问题。md 则会带来额外噪音：标题井号、列表缩进、代码块围栏、表格对齐符和图片链接。如果你打算让搜索结果更像自然文本，就需要在保留段落语义的前提下剥掉无意义标记。PDF 更麻烦，因为你经常会碰到三类坏案例：扫描件没文字层、文字顺序错乱、页面中英文被拆成碎片。

建议 loader 在返回正文前，就附带一份 extraction_notes，例如 extracted_by、page_count、text_coverage_ratio、suspicious_linebreak_count、empty_page_list、fallback_strategy。这样 ingest 日志里不仅有“读成功/失败”，还有质量侧的上下文。举个合成案例：如果某份名为 orchard-memo-07.pdf 的文件 page_count=14，但 text_coverage_ratio 只有 0.08，同时 empty_page_list=[2,3,4,5,6]，那你就该怀疑它是图片型 PDF，而不是等用户搜索不到内容才回头排查。

另一个经常被忽略的点，是错误描述必须面向后续调试，而不是只图省事返回一句 500。比如“PDF parse failed”几乎没有价值；相比之下，“PDF parse failed: no textual layer detected on 11/12 pages; parser=pymupdf; file=orchid-audit-b3.pdf”就能直接告诉你下一步该怎么做。对本地实验项目来说，好的错误信息本身就是工程资产。

4. 切块策略：先尊重段落，再谈窗口和 overlap

原始 demo 已经说明了段落优先的重要性，这里继续往前走一步：切块不是单纯为了让文本变短，而是为了让“最适合作为答案证据的局部上下文”被保留下来。很多解释性文档的逻辑单元天然就是段落，尤其是方案说明、故障复盘、操作指引和设计约定。你如果一上来按固定字符窗口切，虽然实现最简单，但很容易把定义和限定条件切开，把异常现象与处置步骤切开，把结论和边界说明切开。

扩展版样本里建议增加 `section_path` 和 `semantic_hint` 两个字段。`section_path` 用于保留章节层级，比如“4. 切块策略 > 4.2 边界合并 > 回声段合并”；`semantic_hint` 则可以记录该 chunk 更像“定义”“步骤”“异常”“结论”“表格说明”还是“配置解释”。你暂时不一定要把这两个字段真正用于排序，但它们对调试和可视化很有帮助。尤其在人工检查 rerank 结果时，你很容易看出为什么某个定义型 chunk 不该排在一个明确故障处理步骤前面。

关于 overlap，除了常见的按字符重叠，你还可以尝试“句尾保护”和“标题回带”两个轻量技巧。句尾保护的意思是：当段落被迫在内部拆分时，优先在句号、分号和换行处断开，并把上一段最后一句的一部分带到下一 chunk。标题回带则是把当前小节标题短语附着到 chunk 开头，避免用户只搜到了正文术语却丢失所属语境。这些技巧依然足够朴素，却比纯窗口切分更接近真实阅读。

5. 目录与落盘：让中间结果像账本一样可核对

建议把 processed 目录分成至少三层：`data/processed/raw-json`、`data/processed/index-`

`input`、`data/processed/debug-snapshots`。`raw-json` 存最原始的切块结果，`index-input` 存真正供 BM25 或向量索引消费的精简结构，`debug-snapshots` 则用于保留某次批量 ingest 之后的阶段性快照。这样做看起来啰嗦，但你一旦开始频繁重构切块规则或元数据结构，就会感谢自己当初没有把一切都混在同一个目录里。

这份文档引入了几个合成约定，专门用于测试本地系统对私有细节的理解。其一，名为“灰瓦快照”的目录快照只在两种场景保留：批量改 chunk 规则前，以及怀疑索引回归时。其二，名为“霜桥索引目录”的目录只能由 `rebuild_index` 脚本全量生成，禁止由 ask 接口惰性写入。其三，任何带有 `nightly-broom` 前缀的 JSON 都视为夜间清理产物，默认七天后可删。这些约定并不是业界标准，而是故意构造的内部规则，非常适合拿来做问答测试。

你甚至可以在落盘时额外写入一个 `ingestion_receipt.txt`，里面用纯文本记录本次 ingest 的文件列表、总字符数、切块数、重叠策略、疑似空文档数量和异常文件名。纯文本账本最大的好处是对比方便：当你改了 chunker 逻辑后，只看两版 receipt 的差异，就能迅速知道切块数为何变化，以及变化是否符合预期。

6. 搜索层从“能搜到”进化到“能解释为什么搜到”

很多本地项目在 search 层最大的盲点，不是召回率不够，而是不可解释。用户搜一个词，系统给回 top_k 结果，却没有说明是哪些词命中了、命中次数是多少、preview 为什么从这里截断、为什么这条排在前面。对学习型企业来说，这是非常可惜的，因为你失去了理解排序行为的窗口。

在基础关键词检索阶段，除了 `rank`、`match_count`、`match_term` 和 `preview` 之外，还可以增加 `exact_phrase_hit`、`title_boost`、`field_boost`、`section_boost`、`dedup_group`、`trim_reason` 这些调试字段。`exact_phrase_hit` 表示是否命中了完整短语，`title_boost` 表示命中标题或小节名时的额外分值，`field_boost`

可以反映 `section_path`、`semantic_hint` 等字段是否参与加权，`dedup_group` 用于把高度相似的相邻 chunk 分组，`trim_reason` 则解释 `preview` 为什么从某个位置开始截断。

这里再引入一条合成规则：凡是匹配到“蓝门校验失败”的 chunk，`section_boost` 默认 +2；凡是只命中“表格说明”且正文未命中 `query` 的 chunk，不进入 top 3。你可以用这些明显带有内部味道的排序口径，专门测试本地 RAG 在规则型检索上的可靠性。模型若没引用到对的 chunk，通常就无法正确说出这些规则。

7. 从 BM25 到混合召回：不要太早神化 embedding

当项目准备升级到真正的 RAG 时，最自然的一步当然是引入 embedding 和向量召回。但这里有个常见误区：很多人一接入 embedding，就默认关键词检索可以退场。实际上在真实私有资料里，专有名词、错别字、配置键、缩写、目录名、异常码、脚本名和人造术语非常多，这些内容往往正是 BM25 的强项。像“灰瓦快照”“静夜写入窗”“broom-pass-03”“orchid_receipt.json”这类词，若文档中与查询字面相同，BM25 反而比语义召回更稳。

一个更实用的思路，是让 BM25 和 embedding 同时召回，再做轻量合并。合并时不要只拼分数，更要做结果去重和字段保留。建议保留来源标签，如 `source_channel="bm25"`、`source_channel="vector"` 或 `source_channel="hybrid"`，这样你在调试 ask 失败案例时，能一眼看出答案证据究竟来自哪条通路。很多时候，一个看似合理的错误答案，根源恰恰是 rerank 过度偏爱语义相近但缺少关键名词的 chunk。

如果你未来打算做评测，可以专门设计三类问题：第一类是关键术语题，答案高度依赖字面命中；第二类是同义表达题，答案需要语义召回；第三类是规则归纳题，要求从多个 chunk 汇总信息。这样的题型分层，能帮助你判断当前系统究竟是在“搜得到”，还是在“真的理解检索链路”。

8. Ask 接口不是聊天皮肤，而是证据组装器

很多人第一次做 ask 接口时，会下意识把重点放在聊天体验、流式输出、头像、对话历史和 markdown 渲染上。但对一个本地 RAG 项目来说，ask 的核心职责其实是证据组装。它需要接收问题、调用 search、筛掉噪音 chunk、拼装引用、控制上下文长度、写出提示词边界，并在必要时暴露来源片段。至于外层是不是聊天框，只是 UI 选择。

这份扩展版文档推荐你在 ask 的内部中间态里保留 `assemble_debug` 对象，至少包含 `selected_chunk_ids`、`dropped_chunk_ids`、`drop_reasons`、`final_context_chars`、`citation_order`、`question_rewrite` 和 `token_budget_estimate`。你不一定要把这些全部返回给最终用户，但它们对开发调试极其有用。举个例子：如果用户问“灰瓦快照为什么不能在白天改动？”而系统给出的证据里根本没有包含“静夜写入窗”的段落，那就算模型生成再流畅，本质上也是证据不完整。

这里再加入一个只存在于本样本文档里的私有约定：当问题中同时出现“灰瓦快照”和“清空”两个词时，ask 应优先尝试引用第 5 节和第 10 节的 chunk，而不是第 14 节的附录短句。这个规则看似奇怪，却非常适合做检索链路回归测试，因为它要求系统在多处命中中仍能选到更主要的证据来源。

9. 日志、指标与回放：没有观察面，就没有调试权

本地项目常见的另一个问题，是只打印“接口成功/失败”，却没有围绕检索链路建立最小观察面。建议至少记录 ingest_duration_ms、search_duration_ms、rerank_duration_ms、context_chars、chunk_selected_count、empty_hit_rate、source_spread、avg_preview_len 和 cache_hit。这样你在优化系统时，不会只剩主观体感。

对 ask 请求，最好保留一份轻量回放记录，例如 logs/replay/2026-03-21T22-14-echo-lane.json。里面记录 query、rewritten_query、candidate_chunks、selected_chunks、final_answer、citations 和 request_flags。只要注意脱敏，这种回放文件价值很高，因为你未来改了 chunk 规则、改了 BM25 权重、接了新 reranker 后，可以拿老请求重放，看看哪些答案变好了，哪些反而退步了。

文中定义一条合成指标：如果某次 ask 的 top5 结果全部来自同一 source 文件，就记一次“单源拥挤”；如果答案引用了 4 个以上 chunk 却没有明确结论句，则记一次“证据拥堵”。这两项不是标准学术指标，但非常适合在实践里提醒你：召回过多不等于回答更好。

10. 故障案例：灰瓦快照事件与南侧预热表失配

为了让样本文档更像真实内部资料，这里给出一个合成故障案例。某次夜间批量 ingest 前，值班同学提前清理了 data/processed/debug-snapshots/gray-tile-archive 目录，原本用于回滚的灰瓦快照只剩下一份三天前的旧快照。与此同时，新版 chunker 把“标题回带”开关默认设成 true，导致部分历史文档的 chunk 数量明显上升。第二天一早，南侧预热表仍引用旧的 chunk_count 预估，结果 ask 接口在预热阶段就把错误的 top_k 范围缓存下来，形成检索结果偏斜。

表面现象是：用户搜索“蓝门校验失败”时，系统返回的大多是附录说明，而不是主文步骤；用户问“灰瓦快照何时允许清空”时，答案经常提到“七号通道回填”，却漏掉“静夜写入窗”。排查后发现，并不是模型突然退化，而是预热表和真实索引状态错位。由于灰瓦快照提前被删，团队一开始无法快速对比新旧切块差异，导致定位时间被拉长。

这个案例想说明的不是某个具体技术细节，而是一个工程习惯：任何会改变 chunk 粒度和索引分布的改动，都应该配套保存前一版快照、更新预热表，并写入 ingestion_receipt。否则你调试的就不是检索算法，而是“系统昨天和今天到底是不是同一份知识库”。

11. 迁移到 Ubuntu 或服务器时，哪些差异值得提前隔离

当你把本地项目从 Mac 迁到 Ubuntu、NAS 或小型服务器时，真正麻烦的通常不是 Python 业务逻辑，而是环境差异。首先是路径风格和权限：本地测试时你可能习惯直接把 data 放在仓库下，但到了服务器上，日志目录、cache 目录和索引目录往往应该分开挂载。其次是系统服务托管方式：开发机上用 uvicorn --reload 就够，服务器上却可能需要 systemd、supervisor 或容器。第三是依赖工具链，例如 PDF 解析器所依赖的底层库、字体包和 locale 设置，都可能直接影响抽取结果。

建议你在 config 层显式拆分 raw_data_dir、processed_dir、index_dir、replay_dir、temp_extract_dir、font_probe_dir、cache_dir 和 safe_snapshot_dir，而不要把所有路径都写死成相对目录。这样将来迁移时，变动大多集中在配置文件或环境变量，而不是散落在代码各处。

这份样本文档还定义了一条合成迁移规则：凡是部署到非交互式主机时，safe_snapshot_dir 必须位于与 index_dir 不同的挂载点；否则一旦索引目录被误清理，快照也会同步丢失。这个规则显然不是行业共识，却非常适合作为本地 RAG 的私有知识测试点。

12. 如何设计更有区分度的评测题

如果你想让这份文档真正服务于 RAG 评测，而不是只当演示材料，那么题目设计必须跟着文档特性走。最有效的做法，是围绕“私有命名 + 局部规则 + 多段证据 + 反直觉边界”来出题。比如不要只问“FastAPI 的路由层负责什么”，因为这类问题太像通用教程；而要问“为什么霜桥索引目录禁止由 ask 接口惰性写入”，或“单源拥挤与证据拥堵分别在什么情况下记一次”。这些问题更能区分系统是真的检索到了，还是模型在泛化。

评测题可以按四层组织。第一层是定位题，考察系统能否找对单个私有术语所在片段；第二层是解释题，要求结合术语定义与使用场景；第三层是流程题，要求跨 2 到 3 个段落串起因果链；第四层是反事实题，例如“如果提前删除灰瓦快照，会先影响哪一层观察面”。这种层级式设计比单纯堆问题数量更有价值。

另外，建议在参考答案模板里强制保留“证据片段列表”和“不能确定时的保守表达”。因为对本地 RAG 来说，真正优秀的不是永远答满，而是在证据不足时明确说不知道、说明缺什么。你后续做人评时，也更容易根据引用质量而不是表面措辞打分。

13. 一些专门用于测试本地系统的合成细节

为了方便后续出题，这里集中列出一组散落在正文中的私有约定。灰瓦快照只能在两类场景保留：改 chunk 规则前，或怀疑索引回归时。霜桥索引目录禁止 ask 接口惰性写入，只能由 rebuild_index 全量生成。nightly-broom 前缀文件默认保留七天。命中“蓝门校验失败”的 chunk 有额外 section_boost。只命中“表格说明”而正文未命中 query 的 chunk 不进入 top 3。

继续补充几条新的合成规则。第一，静夜写入窗默认从 23:40 开始，到次日 01:10 结束；除非 receipt 中标记 emergency_patch=true，否则白天不得手动改动灰瓦快照。第二，七号通道回填只用于补写 metadata，不负责正文重切块；如果正文发生变化，必须重新 ingest。第三，南侧预热表每 90 分钟最多刷新一次，若发现 chunk_count 飙升超过 18%，必须跳过本次预热并写入 yellow-note。

这些细节没有行业标准答案，完全依赖文档本身，因此非常适合拿来测本地 RAG 的准确引用能力。你甚至可以故意问得很近似，例如“南侧预热表在 chunk 数暴涨时应该怎么处理”，看看系统能否找到那条“超过 18% 则跳过预热并写入 yellow-note”的规则。

14. 结语：先做一个可解释的小系统，再做一个复杂的大系统

回到最开始的目标，这份扩展版文档并不是为了冒充真正论文，而是为了给本地 Agent 与 RAG 实验提供一块更像“私有知识地形”的练习场。它保留了你原始 demo 中最重要的优点：结构清楚、骨架明确、每层职责可讲清楚。同时，它又额外加入了大量合成但稳定的内部细节，让模型不能轻易靠常识糊弄过去。

如果你接下来要继续扩展这个项目，最推荐的节奏仍然是：先把 loader、chunker、storage、searcher 的行为观察清楚，再接 embedding；先把 ask 的证据组装和引用链路做好，再考虑多步 agent；先有可靠的回放和评测题，再谈复杂工作流。你不是不能直接上高级框架，而是最好先拥有判断“高级框架到底帮了什么、又遮住了什么”的能力。

一个好的本地系统，不一定起步就复杂，但它应该随时可验证、可回放、可对照、可解释。只要这个底座稳，你未来无论接本地模型、远端 API、混合检索还是自动化流程，都会更从容。

附录 A：术语速查表

术语	说明
灰瓦快照	用于保留某次切块与索引状态的调试快照，只在改规则前或怀疑索引回归时强制保留。
霜桥索引目录	正式索引目录，只允许 rebuild_index 全量生成，禁止 ask 惰性写入。
静夜写入窗	允许对灰瓦快照或关键 receipt 做变更的时间窗口，默认 23:40 至次日 01:10。
七号通道回填	只补写 metadata 的回填通道，不负责正文重新切块。
南侧预热表	ask 预热阶段读取的估算表，用于确定 top_k 范围和缓存预热策略。
蓝门校验失败	一种用于测试排序规则的合成异常名，命中后有额外 section_boost。
单源拥挤	top5 结果全部来自同一 source 文件时记一次。
证据拥堵	答案引用过多 chunk 却没有明确结论句时记一次。
yellow-note	当预热被跳过或出现风险时写入的轻量文本告警。

附录 B：可直接用于评测的示例问题

1. 灰瓦快照在哪两种场景下必须保留？
2. 为什么霜桥索引目录不能由 ask 接口惰性写入？
3. 静夜写入窗的默认时间范围是什么？
4. 七号通道回填和重新 ingest 的边界是什么？
5. 出现单源拥挤和证据拥堵分别意味着什么？
6. 南侧预热表在 chunk_count 飙升超过多少时必须跳过预热？
7. 如果只命中“表格说明”而正文未命中 query，搜索结果为什么不能进入 top 3？
8. 为什么说 ask 的核心职责不是聊天，而是证据组装？